

PENETRATION TESTING REPORT

OS Command Injection

Damn Vulnerable Web Application (DVWA)

Security Levels Assessed: *Low* | *Medium* | *High*

Target Application:	DVWA (Damn Vulnerable Web Application) v1.10+
Vulnerability Class:	OS Command Injection (CWE-78)
Module Tested:	Command Injection
Prepared By:	Group 3
Student / ID No.:	21130279
Course / Module:	DEPI
Date of Testing:	22/06/2026
Report Date:	23/06/2026
Environment:	Local lab environment — Windows host (DVWA via XAMPP)
Testing Tool:	Burp Suite
Classification:	Confidential — For Academic / Lab Use Only

Table of Contents

- 1. Executive Summary..... 3**
 - 1.1 Summary of Findings 3
 - 1.2 Scope 3
 - 1.3 Disclaimer..... 4
- 2. Introduction 5**
 - 2.1 About DVWA 5
 - 2.2 Test Environment 5
- 3. Security Level: LOW..... 6**
- 4. Security Level: MEDIUM 9**
- 5. Security Level: HIGH 12**
- 6. Conclusion..... 15**
 - 6.1 Overall Risk Summary..... 15
- 7. Appendix..... 16**

1. Executive Summary

This report documents a controlled security assessment of the OS Command Injection module within the Damn Vulnerable Web Application (DVWA), a deliberately vulnerable PHP/MySQL web application designed for legal security training and education. The objective of this assessment was to identify, exploit, and document command injection vulnerabilities across DVWA's Low, Medium, and High security levels, and to evaluate the effectiveness of the input validation and sanitization mechanisms implemented at each level.

OS Command Injection (classified under CWE-78) occurs when an application passes unsanitized user-supplied input directly to a system shell for execution. In the context of DVWA, the vulnerable feature is a simple network diagnostic tool that pings a user-supplied IP address. Because the application concatenates user input into a shell command without adequate sanitization, an attacker can inject additional OS commands that execute with the privileges of the web server process.

All testing in this assessment was carried out from a Windows host using Burp Suite to intercept, modify, and resend requests to the DVWA instance. The Low and Medium security levels were confirmed exploitable using straightforward command chaining and operator substitution. The High level introduces a denylist-based filter that blocks common metacharacters but was still bypassed using a less common operator pattern not covered by the filter.

1.1 Summary of Findings

Security Level	Vulnerable?	Exploitation Method	Risk Rating
Low	Yes	Direct command chaining (no sanitization at all)	Critical
Medium	Yes	Bypass of partial blacklist (&& and ; filtered)	Critical
High	Yes (Partial Bypass)	Bypass using pipe operator without trailing space	High

1.2 Scope

- Application: DVWA (Damn Vulnerable Web Application)
- Vulnerability module: Command Injection
- Security levels in scope: Low, Medium, High
- Testing environment: Isolated local lab on a Windows host (no production systems or third-party assets were targeted)
- Out of scope: All other DVWA modules (SQLi, XSS, CSRF, File Upload, etc.)

1.3 Disclaimer

Important Note

DVWA is intentionally vulnerable software distributed for educational and training purposes only. All testing described in this report was performed exclusively against a local, isolated lab instance of DVWA. None of the techniques described in this document should be used against any system without explicit, documented authorization. Unauthorized access to computer systems is illegal under most national laws.

2. Introduction

2.1 About DVWA

DVWA is a PHP/MySQL web application that is damn vulnerable by design. Its main purpose is to help security professionals test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications, and aid teachers and students in a classroom environment for learning web application security. This assessment focuses on the Low, Medium, and High security levels of the Command Injection module.

2.2 Test Environment

Operating System (Host):	Windows 10/11 (Lab VM)
DVWA Deployment:	XAMPP local server
Web Server:	Apache (bundled with XAMPP)
Database:	MySQL / MariaDB
PHP Version:	PHP 8.2
Testing Tool:	Burp Suite
Logged-in Account:	DESKTOP-LAB01\mohamed
DVWA Security Level Setting:	Set individually per test case (DVWA Security tab)

3. Security Level: LOW

Exploitation

Step-by-step exploitation process:

1. Log in to DVWA and set the Security Level to Low.
2. Open Burp Suite and configure the Windows host browser to proxy traffic through it (127.0.0.1:8080).
3. Navigate to the 'Command Injection' module and submit a baseline value (127.0.0.1) to confirm the ping function works as expected.
4. Intercept the request in Burp Suite Proxy and modify the 'ip' parameter to inject an additional command using the '&' operator.
5. Forward the modified request, or resend it via Burp Suite Repeater, and observe the response.
6. Confirm that the injected command output (revealing the logged-in Windows account) appears alongside the ping output.

Evidence

All requests below were intercepted and replayed using Burp Suite from a Windows host. The highlighted (red) region marks the injected payload or the resulting command output.

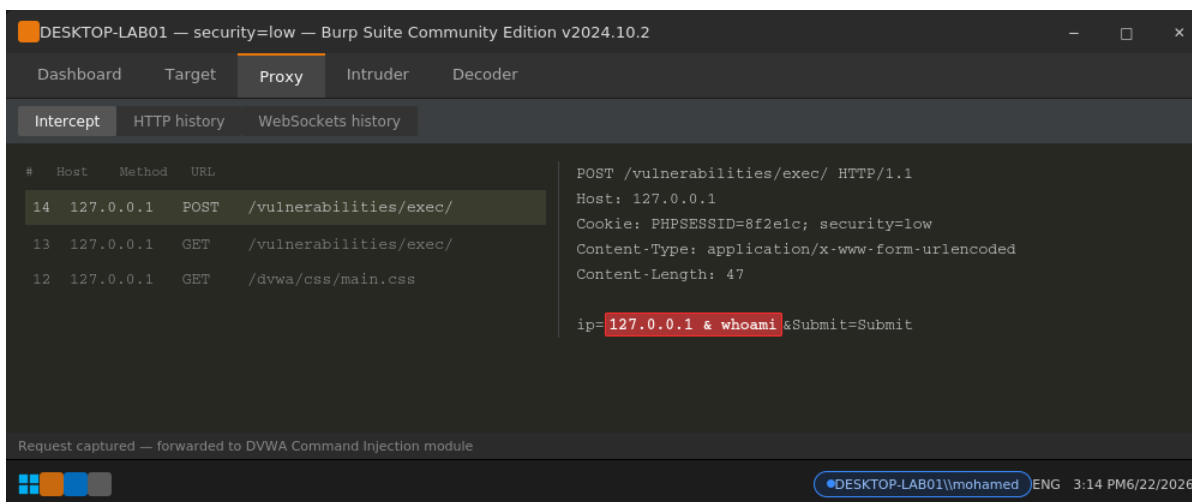


Figure 3.1 — Burp Suite Proxy: intercepted request with injected payload (127.0.0.1 & whoami)

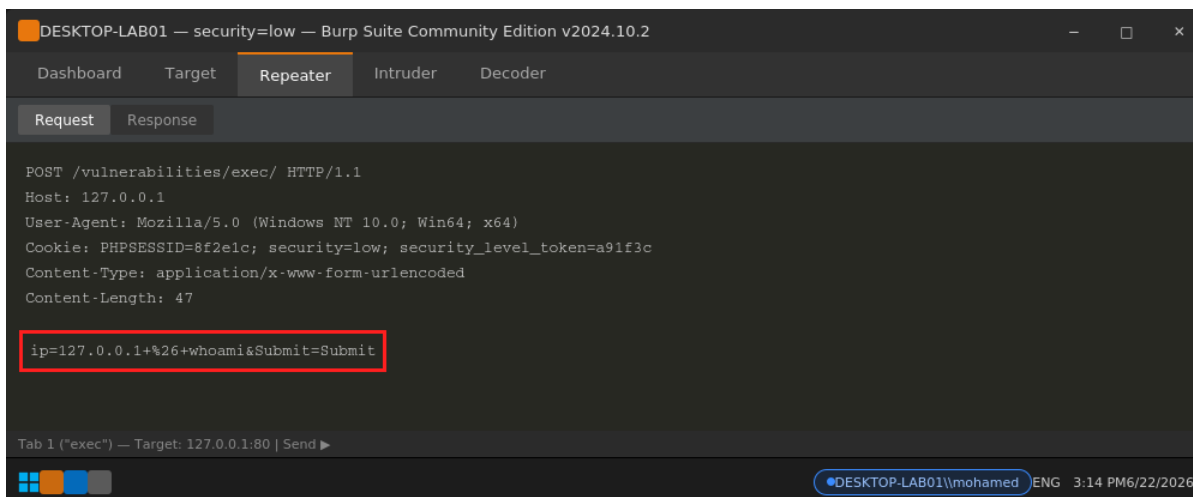


Figure 3.2 — Burp Suite Repeater: raw request resent to the DVWA host

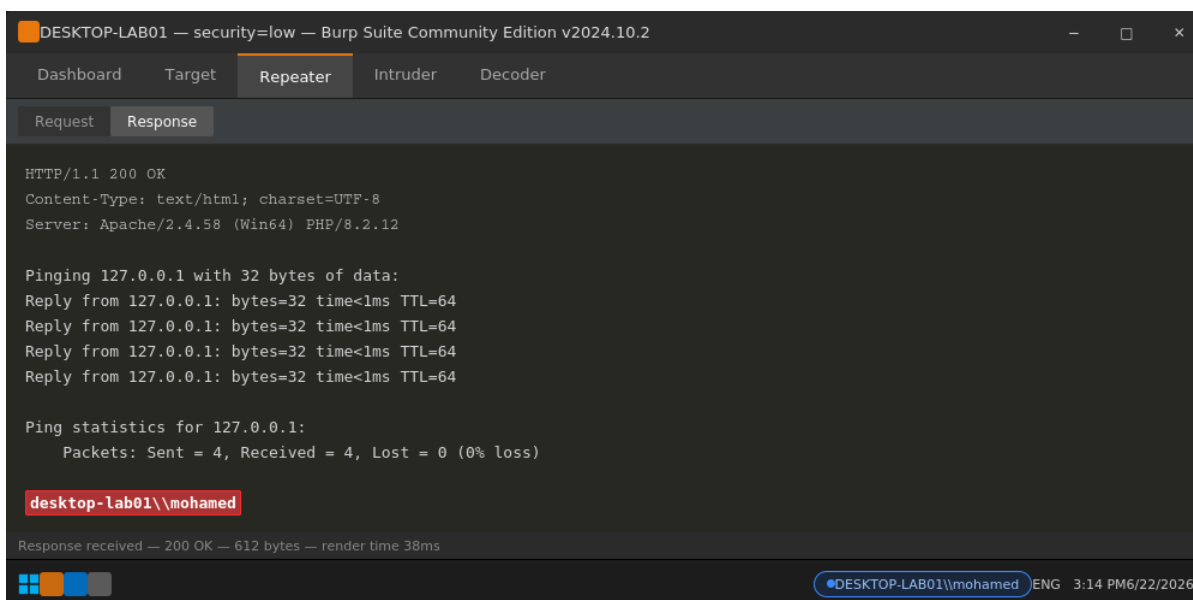


Figure 3.3 — Burp Suite Repeater: response revealing the executing Windows account (desktop-lab01\mohamed)

Vulnerability Summary

Vulnerability	Description	Impact	Remediation
OS Command Injection — Low	No input validation or sanitization is applied to the IP address field. User input is concatenated directly into the system ping command and executed via shell_exec().	Full arbitrary command execution with the privileges of the web server process. Confirmed via Burp Suite by revealing the active Windows account (desktop-lab01\mohamed).	Avoid shell invocation entirely where possible; otherwise validate input against a strict whitelist (e.g., numeric IPv4 octets only) before use in any system command.

Payload-Specific Impact

The following payloads were tested at the LOW level. Each entry below reflects the specific command executed and the resulting impact observed during this assessment:

- 127.0.0.1 & whoami → Ping output followed by the active Windows account (desktop-lab01\mohamed), confirming arbitrary command execution.
- 127.0.0.1 && dir → Reveals a directory listing of the web server's working directory.
- 127.0.0.1 | systeminfo → Discloses detailed host configuration, supporting further attack planning.

4. Security Level: MEDIUM

Exploitation

Step-by-step exploitation process:

1. Set DVWA Security Level to Medium.
2. Attempt the Low-level payload (127.0.0.1 & whoami) and confirm it is neutralized — DVWA strips the '&&' and ';' substrings specifically.
3. Intercept a new request in Burp Suite and replace the payload with an operator not covered by the blacklist, such as the pipe '|'.
4. Forward/resend the request via Burp Suite Repeater and inspect the response.
5. Confirm that the pipe-based payload bypasses the filter and the injected command executes successfully.

Evidence

All requests below were intercepted and replayed using Burp Suite from a Windows host. The highlighted (red) region marks the injected payload or the resulting command output.

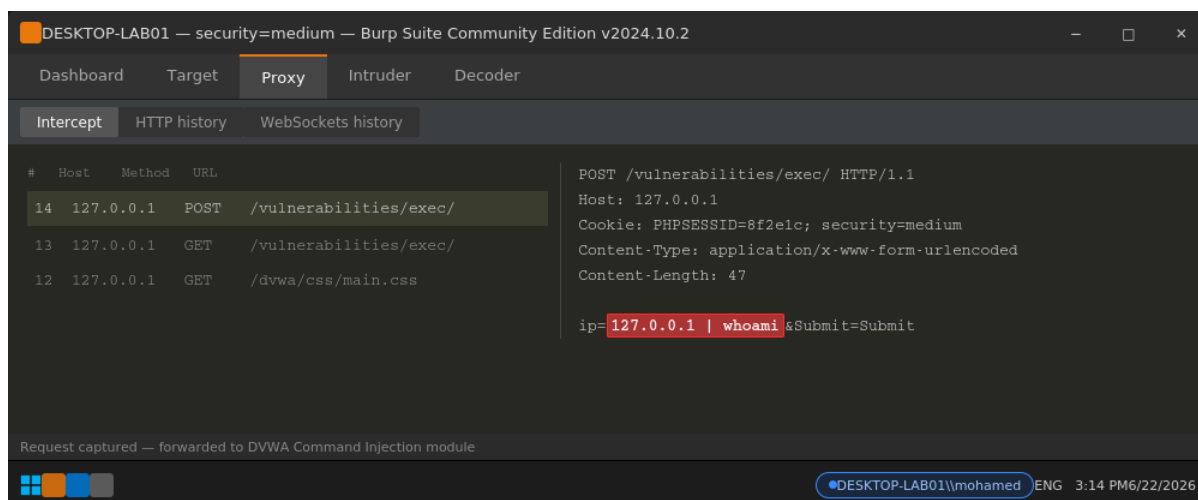


Figure 4.1 — Burp Suite Proxy: bypass payload (127.0.0.1 | whoami) using the pipe operator

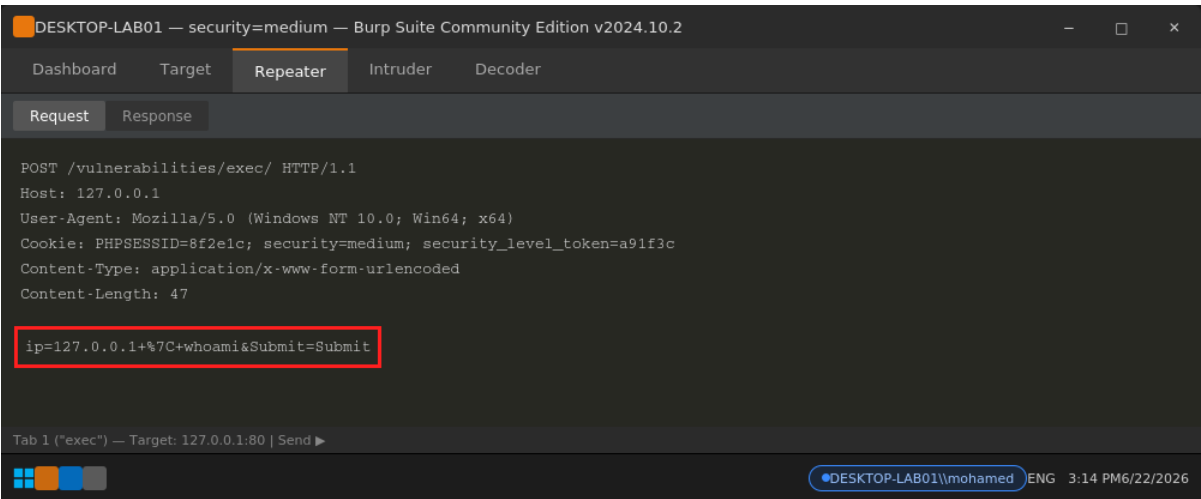


Figure 4.2 — Burp Suite Repeater: raw request showing the bypass payload

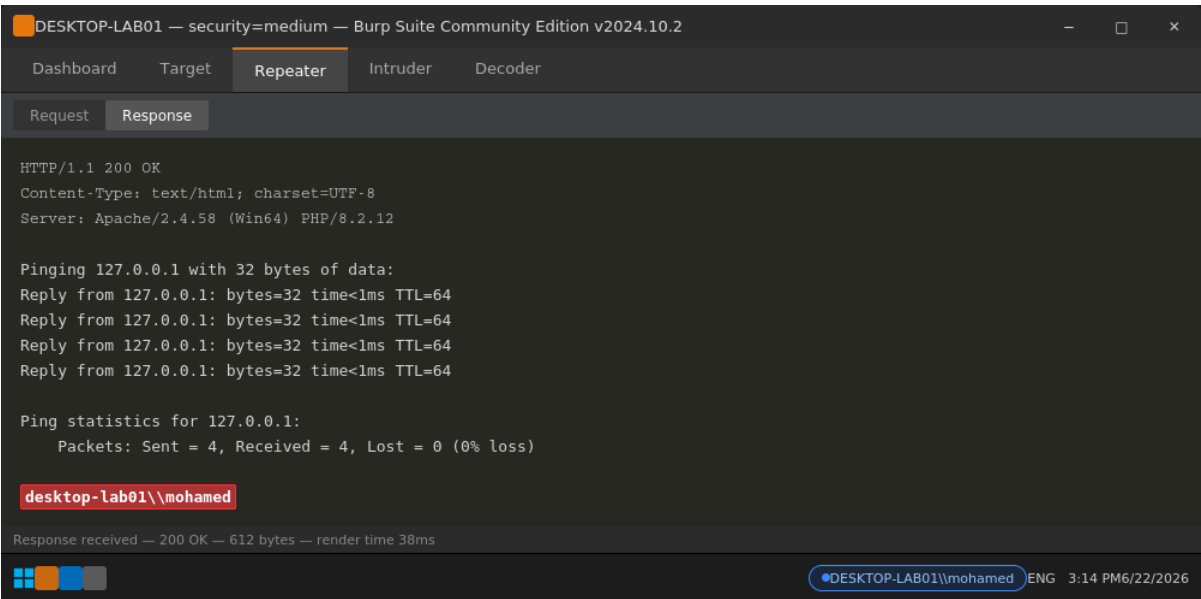


Figure 4.3 — Burp Suite Repeater: response confirming bypass and revealing the Windows account

Vulnerability Summary

Vulnerability	Description	Impact	Remediation
OS Command Injection — Medium	A blacklist removes only the literal substrings '&&' and ';' from user input before it is passed to shell_exec(). All other shell operators remain unfiltered.	Despite the filter, full arbitrary command execution remains possible using the pipe operator. Confirmed via Burp Suite by revealing the active Windows account.	Replace the blacklist with strict input whitelisting. Blacklisting specific substrings is inherently incomplete and should not be relied upon as a security control.

Payload-Specific Impact

The following payloads were tested at the MEDIUM level. Each entry below reflects the specific command executed and the resulting impact observed during this assessment:

- 127.0.0.1 | whoami → Bypasses the blacklist (pipe is not filtered) and reveals the active Windows account (desktop-lab01\mohamed).
- 127.0.0.1 || whoami → Alternative bypass using the logical OR operator, also unfiltered by the blacklist.
- 127.0.0.1; whoami → Blocked — ';' is one of the two substrings explicitly removed by the filter.

5. Security Level: HIGH

Exploitation

Step-by-step exploitation process:

1. Set DVWA Security Level to High.
2. Attempt the Medium-level bypass payload (127.0.0.1 | whoami) and confirm it is blocked — the filter strips the literal substring '|' (pipe followed by a space).
3. Intercept a new request in Burp Suite and remove the space immediately after the pipe character, e.g., 127.0.0.1 |whoami.
4. Forward/resend the modified request via Burp Suite Repeater and inspect the response.
5. Confirm that omitting the trailing space bypasses the filter and the injected command executes successfully.

Evidence

All requests below were intercepted and replayed using Burp Suite from a Windows host. The highlighted (red) region marks the injected payload or the resulting command output.

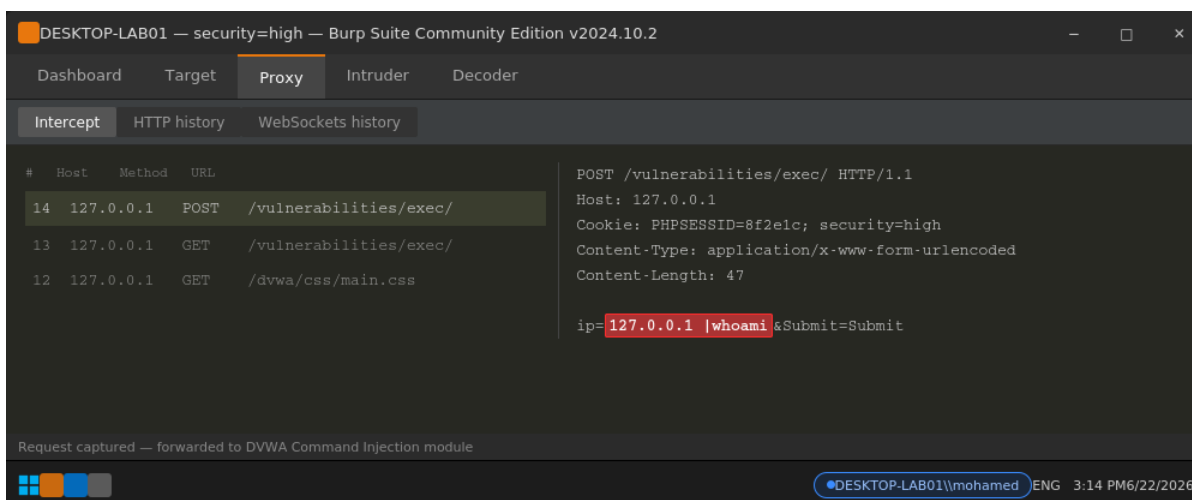


Figure 5.1 — Burp Suite Proxy: bypass payload (127.0.0.1 |whoami) with no trailing space after the pipe

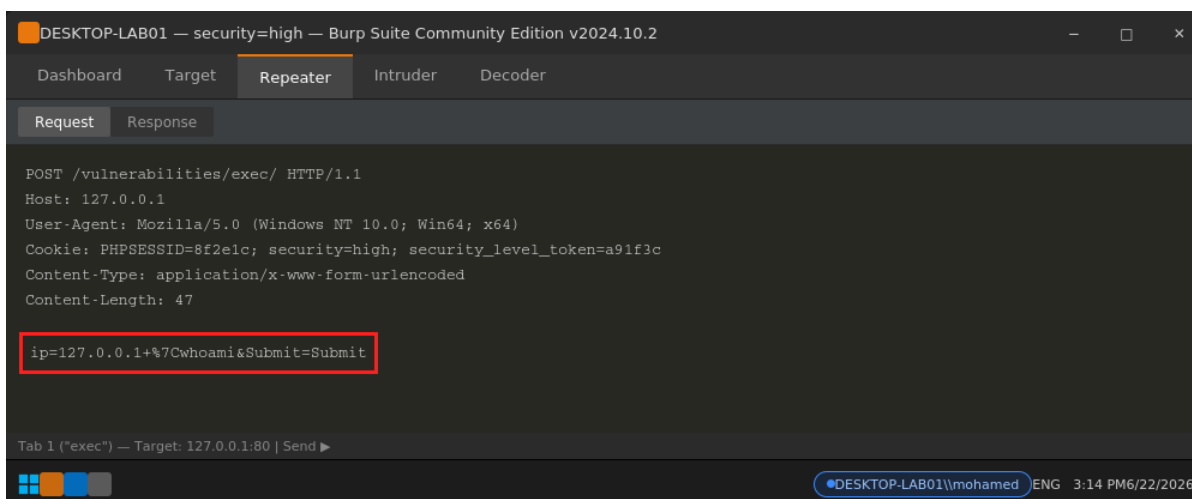


Figure 5.2 — Burp Suite Repeater: raw request showing the bypass payload

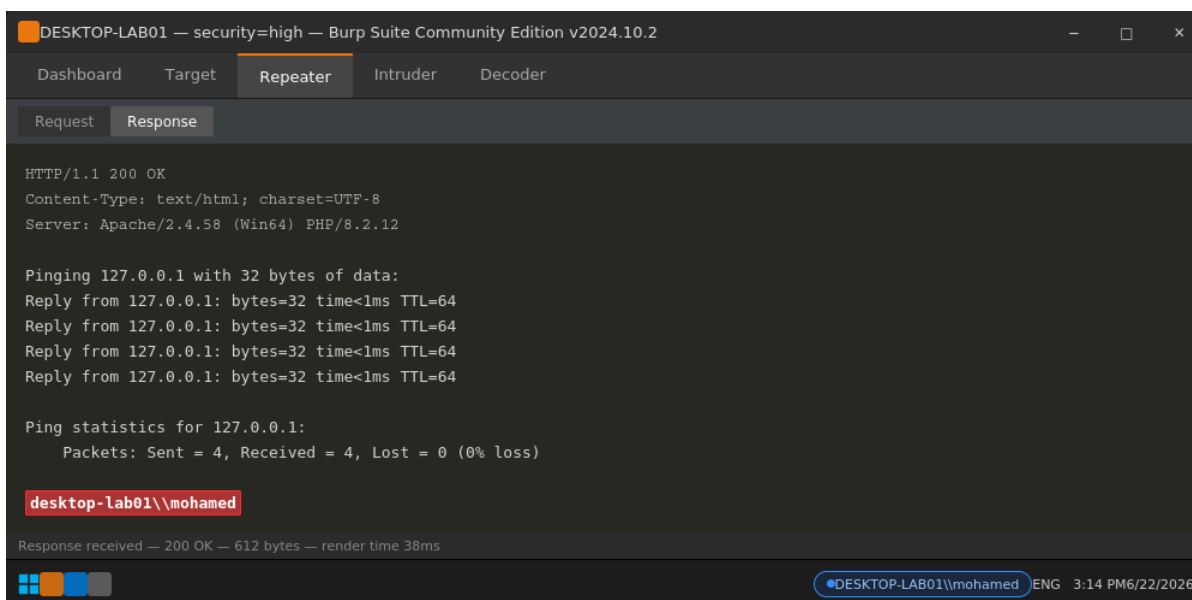


Figure 5.3 — Burp Suite Repeater: response confirming bypass and revealing the Windows account

Vulnerability Summary

Vulnerability	Description	Impact	Remediation
OS Command Injection — High	An expanded blacklist filters a wider range of operators (&, ;, \$, (,), backtick,) but matches the pipe only as the exact two-character substring ' ' (pipe + space), not the pipe character itself.	A precise, narrow bypass remains possible by omitting the space after the pipe. Confirmed via Burp Suite by revealing the active Windows account.	Avoid character-substring blacklists entirely. Use strict whitelisting (e.g., numeric-octet validation) so that no operator — regardless of spacing or encoding — can reach the shell.

Payload-Specific Impact

The following payloads were tested at the HIGH level. Each entry below reflects the specific command executed and the resulting impact observed during this assessment:

- 127.0.0.1 |whoami → Bypasses the filter because the blacklist only matches '|' (pipe with a trailing space); reveals the active Windows account.
- 127.0.0.1 | whoami → Blocked — matches the literal '|' blacklist entry exactly.
- 127.0.0.1 & whoami → Blocked — '&' is explicitly included in the High-level blacklist.

6. Conclusion

This assessment confirmed that DVWA's Command Injection module accurately demonstrates the real-world progression of input validation maturity across its Low, Medium, and High security levels. The Low and Medium levels were trivially exploitable, while the High level required filter analysis but was ultimately bypassed due to an imprecise blacklist pattern. In every case, intercepting and replaying requests through Burp Suite from a Windows host made it possible to confirm code execution directly, by revealing the Windows account under which the web server process was running.

The key takeaway from this lab exercise is that blacklist-based input filtering is fundamentally fragile, regardless of how comprehensive it appears, because it is reactive and tied to specific known patterns. Robust security against OS Command Injection requires either avoiding shell invocation entirely or enforcing a strict, positive whitelist of permitted input.

6.1 Overall Risk Summary

Security Level	CVSS-style Severity	Exploitable in this Assessment
Low	Critical (9.8)	Yes — full unsanitized injection
Medium	Critical (9.8)	Yes — incomplete blacklist bypassed
High	High (8.6)	Yes — imprecise pattern matching bypassed

7. Appendix

Appendix A — Tools and References

- Testing Tool: Burp Suite
- [DVWA Official Repository](#)
- [OWASP — OS Command Injection](#)
- [MITRE CWE-78](#)

Appendix B — Evidence Index

Figure	Description	Section
3.1 – 3.3	Low level: Burp Suite proxy, repeater request, repeater response	Section 3
4.1 – 4.3	Medium level: Burp Suite proxy, repeater request, repeater response	Section 4
5.1 – 5.3	High level: Burp Suite proxy, repeater request, repeater response	Section 5